



PES

UNIVERSITY

CELEBRATING 50 YEARS

PROGRAMMING WITH PYTHON

Lekha A

Computer Applications

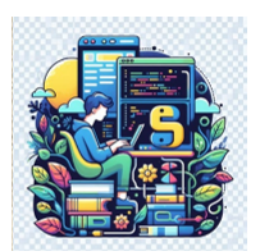
PROGRAMMING WITH PYTHON

Object-Oriented Programming (OOP) and Advanced Concepts

Composition

Lekha A

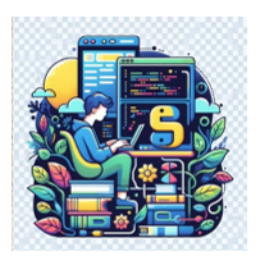
Computer Applications



PROGRAMMING WITH PYTHON

Composition

- Composition is a concept that models a has a relationship.
- It enables creating complex types by combining objects of other types.
- This means that a class Composite can contain an object of another class Component.
- This relationship means that a Composite has a Component.



PROGRAMMING WITH PYTHON

Composition

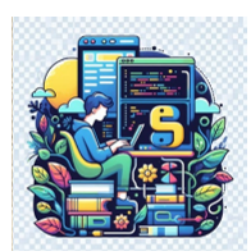
- Composition is the act of collecting several objects together to create a new one.
- Composition is usually a good choice when one object is part of another object.



PROGRAMMING WITH PYTHON

Composition

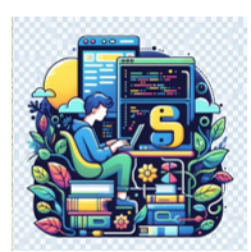
- Example
 - A **Laptop** is composed of several key components — such as the **Processor, Memory, Storage, Battery, Display, and Keyboard.**
 - Each of these components can further be broken down into smaller parts.



PROGRAMMING WITH PYTHON

Composition

- The Laptop object can provide the interface required by a user, while also giving access to its component parts, which offers the deeper level of abstraction suitable for a repair associate.
- Those component parts can, of course, be further broken down if the repair associate needs more information to diagnose a problem or tune the engine.



PROGRAMMING WITH PYTHON

Composition

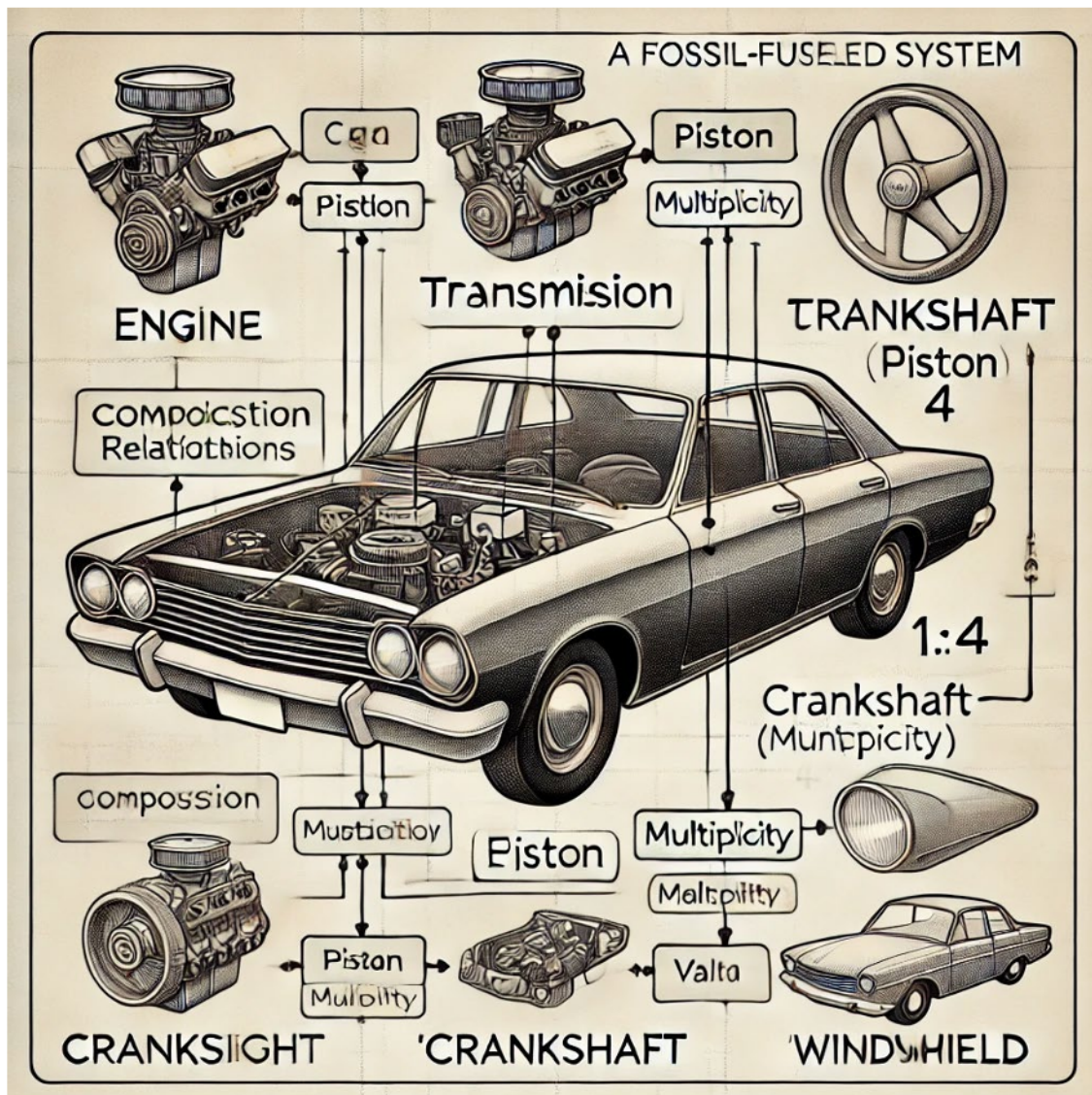
Laptop

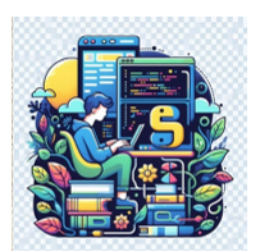
- Processor
 - Cores (8)
 - Cache
 - Threads (16)
- Memory (RAM)
 - Modules (2)
 - Capacity (16 GB)
- Storage
 - SSD
 - Controller
- Battery
 - Cells (4)
 - Capacity (56 Wh)
- Display
 - Screen Panel
 - Backlight
- Keyboard
 - Keys (104)
 - Backlight
- Touchpad



PROGRAMMING WITH PYTHON

Composition

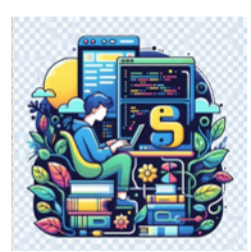




PROGRAMMING WITH PYTHON

Composition

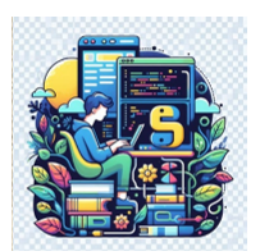
- Consider the design of a computerized chess game.
 - As a basic, high-level analysis, a *game* of chess is *played* between *two players*, using a chess set featuring a *board* containing sixty-four *positions* in an 8x8 grid.
 - The board can have two sets of sixteen *pieces* that can be *moved*, in alternating *turns* by the two players in different ways.



PROGRAMMING WITH PYTHON

Composition

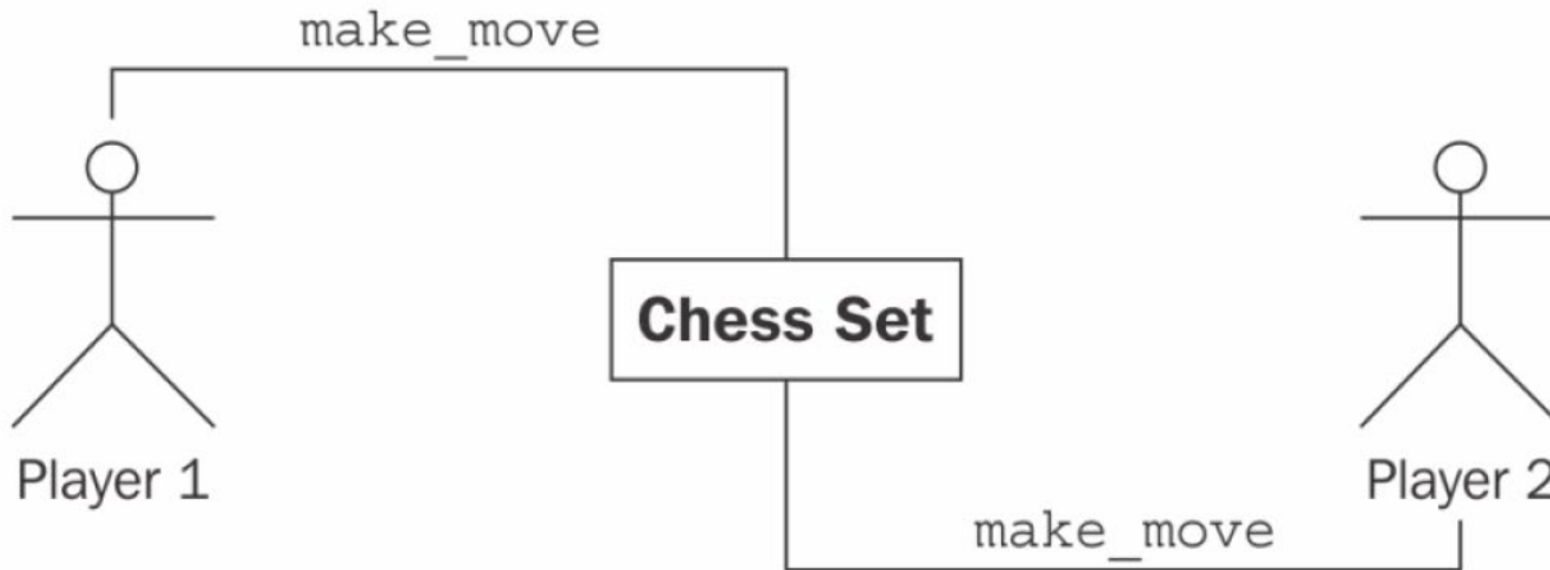
- Each piece can **take** other pieces.
- The board will be required to **draw** itself on the computer **screen** after each turn.
 - *Objects*
 - **Methods**

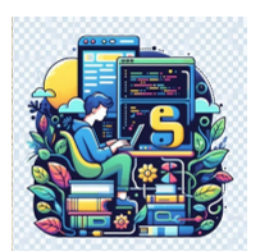


PROGRAMMING WITH PYTHON

Composition

- At the highest level of abstraction possible.
- Two players interacting with a Chess Set by taking turns making moves

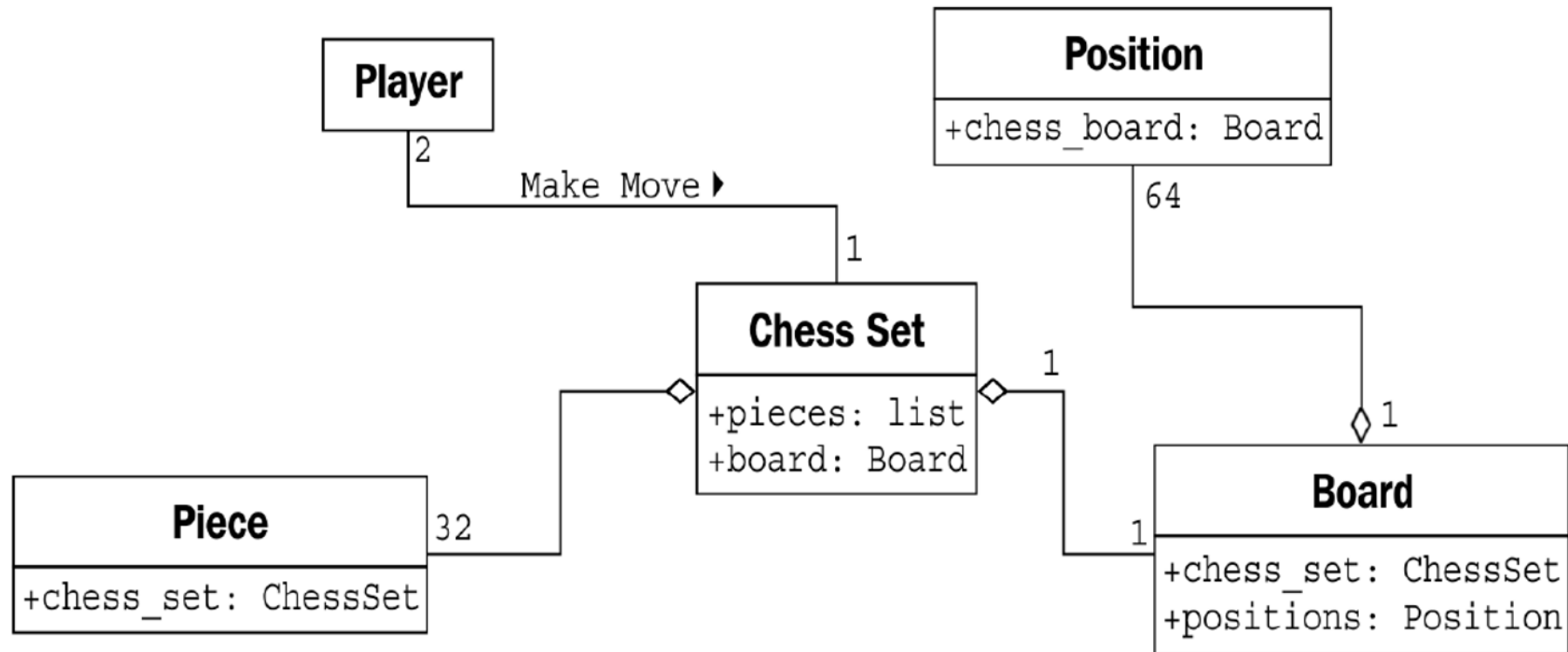


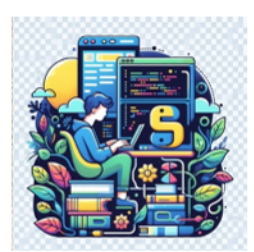


PROGRAMMING WITH PYTHON

Composition

- The chess set, then, is composed of a board and 32 pieces.
- The board further comprises 64 positions.

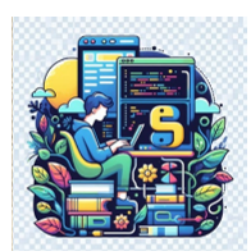




PROGRAMMING WITH PYTHON

Composition

- Composition is represented through a line with a diamond at the composite class pointing to the component class.
- The composite side can express the cardinality of the relationship.
- The cardinality indicates the number or valid range of Component instances the Composite class will contain.



PROGRAMMING WITH PYTHON

Composition

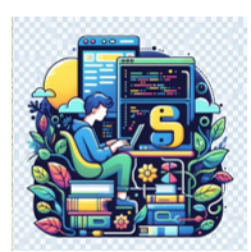
```
class Piece:
    def __init__(self, color):
        self.color = color

class Pawn(Piece):
    def __init__(self, color):
        super().__init__(color)
        self.type = "Pawn"

class Board:
    def __init__(self):
        self.grid = [[None for _ in range(8)] for _ in range(8)]

    def initialize_board(self):
        # Initialize white pieces
        self.grid[1] = [Pawn("white") for _ in range(8)]
        # Initialize black pieces
        self.grid[6] = [Pawn("black") for _ in range(8)]

    def print_board(self):
        for row in self.grid:
            print([piece.type if piece else " - " for piece in row])
```



PROGRAMMING WITH PYTHON

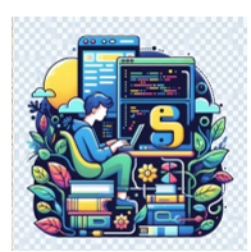
Composition

```
class Game:
    def __init__(self):
        self.board = Board()
        self.board.initialize_board()

    def play(self):
        print("Initial Board:")
        self.board.print_board()
```

Initial Board:

```
[' - ', ' - ', ' - ', ' - ', ' - ', ' - ', ' - ', ' - ']  
['Pawn', 'Pawn', 'Pawn', 'Pawn', 'Pawn', 'Pawn', 'Pawn', 'Pawn']  
[' - ', ' - ', ' - ', ' - ', ' - ', ' - ', ' - ', ' - ']  
[' - ', ' - ', ' - ', ' - ', ' - ', ' - ', ' - ', ' - ']  
[' - ', ' - ', ' - ', ' - ', ' - ', ' - ', ' - ', ' - ']  
[' - ', ' - ', ' - ', ' - ', ' - ', ' - ', ' - ', ' - ']  
['Pawn', 'Pawn', 'Pawn', 'Pawn', 'Pawn', 'Pawn', 'Pawn', 'Pawn']  
[' - ', ' - ', ' - ', ' - ', ' - ', ' - ', ' - ', ' - ']
```

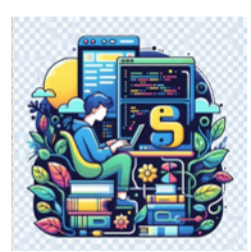
PROGRAMMING WITH PYTHON

Composition

Creating a class for Processor

```
class Processor:
    def __init__(self, brand, cores):
        self.brand = brand
        self.cores = cores

    def process(self):
        print(f"Processor running with {self.cores} cores ({self.brand})")
```



PROGRAMMING WITH PYTHON

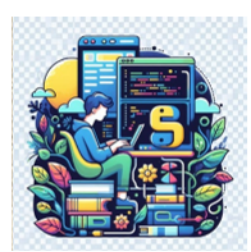
Composition

Creating a class for Battery

```
class Battery:
    def __init__(self, capacity):
        self.capacity = capacity # in mAh

    def charge(self):
        print(f"Battery charging... Capacity: {self.capacity}mAh")

    def discharge(self):
        print("Battery discharging...")
```



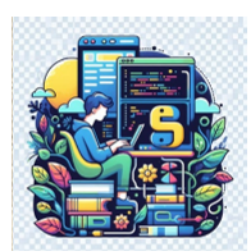
PROGRAMMING WITH PYTHON

Composition

- Creating a class for Screen

```
class Screen:
    def __init__(self, size):
        self.size = size # in inches

    def display(self, content):
        print(f"Displaying on {self.size}-inch screen: {content}")
```



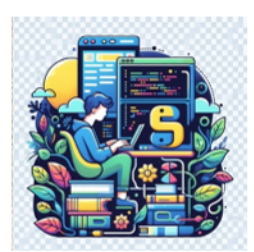
PROGRAMMING WITH PYTHON

Composition

- Creating a class for Keyboard

```
class Keyboard:
    def __init__(self, type_):
        self.type = type_

    def type_key(self, key):
        print(f"Key '{key}' pressed on {self.type} keyboard")
```



PROGRAMMING WITH PYTHON

Composition

Creating a class for Laptop using composition

```
class Laptop:
    def __init__(self, brand, model):
        self.brand = brand
        self.model = model

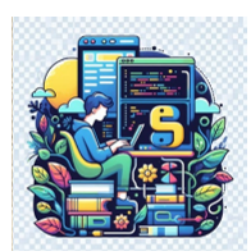
    # Composing objects
    self.processor = Processor("Intel i7", 8)
    self.battery = Battery(6000)
    self.screen = Screen(15.6)
    self.keyboard = Keyboard("Backlit")

    def power_on(self):
        print(f"{self.brand} {self.model} powering ON...")
        self.processor.process()

    def show_display(self, message):
        self.screen.display(message)

    def charge_laptop(self):
        self.battery.charge()

    def use_keyboard(self, key):
        self.keyboard.type_key(key)
```

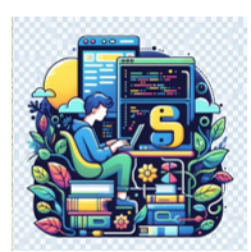


PROGRAMMING WITH PYTHON

Composition

- `my_laptop = Laptop("Dell", "XPS 15")`

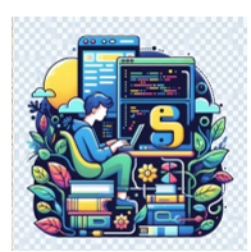
```
my_laptop.power_on()  
my_laptop.show_display("Welcome to your system!")  
my_laptop.use_keyboard("Enter")  
my_laptop.charge_laptop()
```



PROGRAMMING WITH PYTHON

Composition

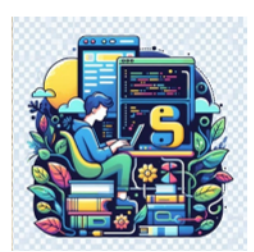
```
Dell XPS 15 powering ON...  
Processor running with 8 cores (Intel i7)  
Displaying on 15.6-inch screen: Welcome to your system!  
Key 'Enter' pressed on Backlit keyboard  
Battery charging... Capacity: 6000mAh
```

PROGRAMMING WITH PYTHON

Reasons for Composition

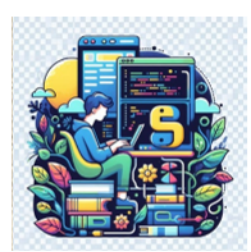
- Promoting Modularity
 - When there is a need to build objects that have clear, independent components, composition helps create modular and reusable code.
 - It allows encapsulation of functionality into smaller, manageable classes.



PROGRAMMING WITH PYTHON

Reasons for Composition

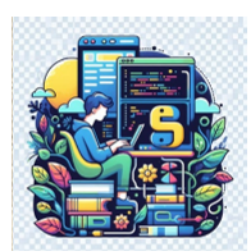
- Flexibility and Reusability
 - Composition enables flexibility in constructing objects by assembling different components.
 - It allows reuse of existing classes without directly inheriting their behavior.



PROGRAMMING WITH PYTHON

Reasons for Composition

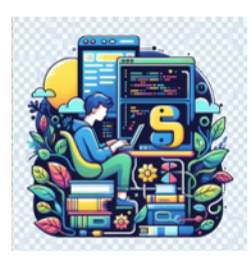
- Avoiding Tight Coupling
 - If there isn't a clear "is-a" relationship between classes (as in inheritance), but rather a "has-a" relationship, composition is a better choice.
 - It avoids creating overly complex or tightly coupled class hierarchies.



PROGRAMMING WITH PYTHON

Reasons for Composition

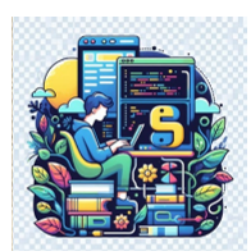
- Managing Complexity
 - When dealing with complex systems, composition allows the break down of functionality into smaller, more manageable parts.
 - This makes code easier to understand, maintain, and extend.



PROGRAMMING WITH PYTHON

Reasons for Composition

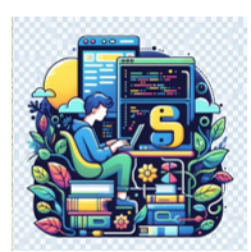
- Changing Behavior Dynamically
 - With composition, the behavior of an object can be changed dynamically at runtime by swapping out its components with different ones.



PROGRAMMING WITH PYTHON

Reasons for Composition

- Dependency Injection
 - Composition is useful for implementing dependency injection, where objects rely on interfaces or abstractions rather than concrete implementations.
 - This allows for more flexible and testable code.



PROGRAMMING WITH PYTHON

Recap

- Composition
- Examples
- Scenarios for Composition



THANK YOU

Lekha A
Computer Applications